Beginner Level

1. What is Jest, and why is it commonly used in React projects?
   Jest is a JavaScript testing framework created by Facebook, and it is very commonly used in React projects.It is popular because it is fast, easy to use, and has built-in features like test runners, mocking, and snapshot testing.React developers prefer Jest because it integrates smoothly with React applications without extra setup.

2. How do you install Jest in a Node.js project?
   npm install --save-dev jest

3. Write a simple Jest test case to check if 2 + 2 = 4.
   ```
   test('adds 2 + 2 = 4', () => {
       expect(2 + 2).toBe(4);
   });
   ```

4. Explain the difference between test() and it() in Jest.
   The test and it functions in Jest serve the same purpose but differ in their naming conventions and how they impact the readability of test descriptions. In Jest, the it() function defines individual test cases, representing a single unit of code you want to verify. It is an alias for the test() function, so we can use either interchangeably, depending on the preference.

5. What does the expect() function do in Jest?
   The expect() function is used to create assertions in Jest.It compares the actual output of the code with the expected value, which is then verified using matchers.

6. What is a "matcher" in Jest? Give 3 examples
   A matcher is a function that lets you check different conditions in your test. For example, .toBe() checks strict equality, .toEqual() checks deep equality like objects and arrays, and .toContain() checks if an array or string contains a value.

7. Write a Jest test to check if a string "Hello World" contains "World".
   ```
   test('string contains World', () => {
     expect("Hello World").toContain("World");
   });
   ```

8. How do you run all Jest tests in a project?
   npm test

9. What is the default test file naming convention used by Jest?
   .test.js

10. How do you check if a function throws an error using Jest?
    ```
    function throwError() { throw new Error("Error!"); }
    test('throws error', () => {
      expect(() => throwError()).toThrow("Error!");
    });
    ```

Intermediate Level

11. What is the purpose of beforeEach() and afterEach() in Jest?
    The `beforeEach()` function runs before every test case and is mainly used to set up test data or environment.The `afterEach()` function runs after every test case and is used for cleaning up or resetting values after each test.

12. Write a Jest test to check if an array [1, 2, 3] contains 2.
```
test('array contains 2', () => {
  expect([1, 2, 3]).toContain(2);
});
```

13. How do you test asynchronous functions (using promises) in Jest?
    We can test asynchronous functions using either `async/await` or by returning a promise.

14. Write a Jest test to check if a function resolves to "success".
```
test('resolves to success', async () => {
  await expect(Promise.resolve("success")).resolves.toBe("success");
});
```

15. How do you use jest.fn()? Give an example.
    `jest.fn()` is used to create a mock function.For example, `const mockFn = jest.fn();` creates a function that can be tested, and you can check if it was called with the right arguments.

16. What is the difference between toBe() and toEqual()?
    The `toBe()` matcher checks strict equality using `===`, so it is best for primitive values.The `toEqual()` matcher checks deep equality, which means it works well for arrays and objects.

17. How do you mock a function using Jest?
    jest.fn()

18. Write a Jest test for an API function that calls fetch().
```
global.fetch = jest.fn(() =>
  Promise.resolve({ json: () => Promise.resolve({ data: "ok" }) })
);
test('fetch api', async () => {
  const res = await fetch();
  const json = await res.json();
  expect(json.data).toBe("ok");
});
```

19. Explain how to test React components with @testing-library/react and Jest.
    We render the component using the `render()` function from `@testing-library/react`.Then, we query elements in the DOM using `screen.getByText()` or other methods.We can simulate actions like clicks with `fireEvent` or `userEvent`.Finally, we use Jest's `expect()` function to assert the expected behavior.

20. What is the difference between jest.spyOn() and jest.fn()?
    `jest.fn()` creates a brand new mock function that we can test in isolation. `jest.spyOn()` allows us to spy on an existing object's method, so we can check if it was called, but we can also call its real implementation if needed.

Advanced Level

21. How do you test time-dependent code with Jest timers (setTimeout, setInterval)?
    We use fake timers by calling `jest.useFakeTimers()`.Then we can control time in tests using `jest.advanceTimersByTime()` to simulate setTimeout or setInterval execution.

22. How do you reset mocks between tests?
    We reset mocks using `jest.resetAllMocks()` inside an `afterEach()` block so that every test starts fresh.

23. Write a Jest test to simulate a button click in a React component.

```
import { render, fireEvent } from "@testing-library/react";
import Button from "./Button";
test('button click', () => {
  const { getByText } = render(<Button />);
  fireEvent.click(getByText("Click Me"));
  expect(getByText("Clicked")).toBeInTheDocument();
});
```

24. How do you mock a module in Jest?
    We can use `jest.mock()` to replace an entire module with a mock implementation.

25. Explain how snapshot testing works in Jest.
    A typical snapshot test case renders a UI component, takes a snapshot, then compares it to a reference snapshot file stored alongside the test. The test will fail if the two snapshots do not match: either the change is unexpected, or the reference snapshot needs to be updated to the new version of the UI component.

26. What's the difference between jest.mock() and jest.requireActual()?
    - `jest.mock()` replaces a module with a mocked version.
    - `jest.requireActual()` is used inside a mock to import the real implementation of a module when you still want to use some parts of it.

27. How do you configure Jest for TypeScript projects?
    install `jest`, `ts-jest`, and `@types/jest`, set `preset: "ts-jest"` in `jest.config.js`, configure `tsconfig.json`, and we can run Jest smoothly with TypeScript.

28. How can you run only a specific test file in Jest?
    npx jest filename.test.js

29. How do you measure code coverage with Jest?
    npx jest –coverage

30. How do you mock axios calls in Jest?

```
jest.mock('axios')
```